

Proiectarea setului de instrucțiuni

Caracteristicile procesorului

- RISC – *Reduced Instruction Set Computer*
- CISC – *Complex Instruction Set Computer*

Utilizarea memoriei și a registrelor

Numărul registrelor, un compromis între cerințe contradictorii (exemplu):

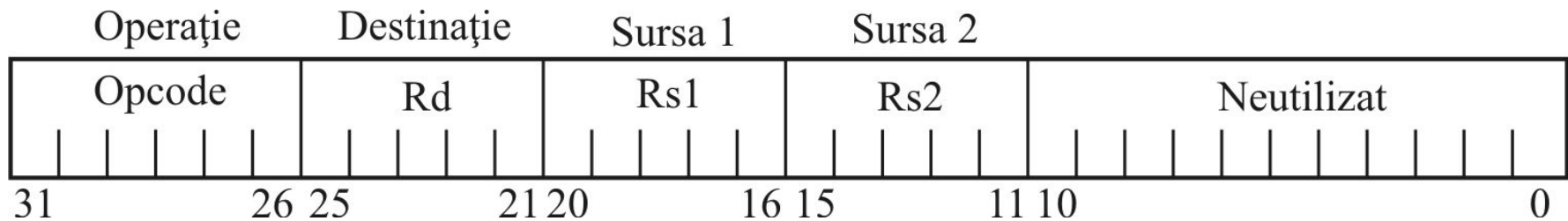
- un număr cât mai mare de registre reprezintă optimul pentru compilator
- comutarea contextului devine tot mai lentă cu creșterea numărului de registre.

Lungimea instrucțiunilor și a operanzilor

Lungimea adreselor de memorie

Formatul instrucțiunii la procesoarele RISC

1. Formatul registru – registru – registru (R-R-R)



Exemplu de instrucțiune registru – registru – registru:

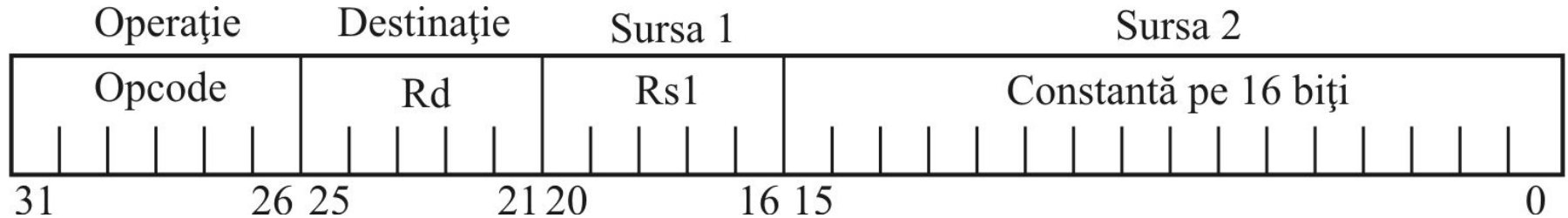
ADD R1,R4,R5 ; R1 = R4 + R5

Utilizând R0, care reprezintă constanta zero, putem crea o instrucțiune *“move”*:

ADD R2,R3,R0 ; R2 = R3 (+0)

Formatul instrucțiunii la procesoarele RISC

2. Formatul registru – registru – constantă (R-R-I)



Exemplu de instrucțiune registru – registru – constantă (registru – registru – imediat):

ADD R4,R5,64 ; R4 = R5 + 64

Toate instrucțiunile din clasa R-R-R au instrucțiuni corespondente în clasa R-R-I

Formatul instrucțiunii la procesoarele RISC

3. Formatul registru – memorie (R-M)

În acest format se încadrează instrucțiunile “*load*” și respectiv “*store*”:

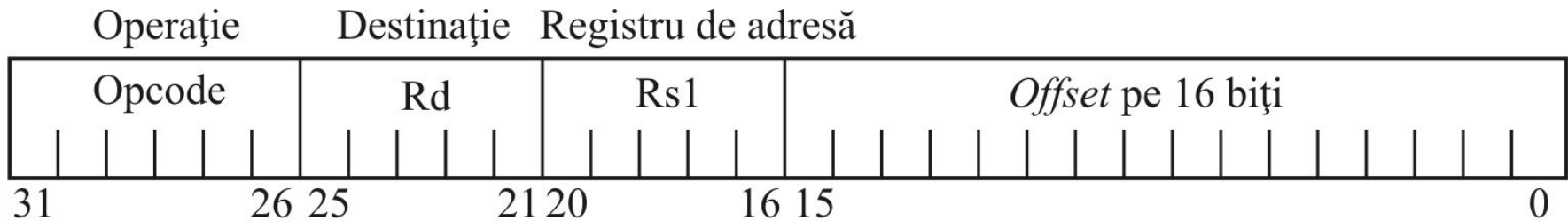
LD R1,200[R8] ; conținutul locației de memorie adresată
; de R8 + 200 este încărcat în R1.

ST 16[R3],R4 ; R4 este copiat în locația de memorie ;
adresată de R3 + 16.

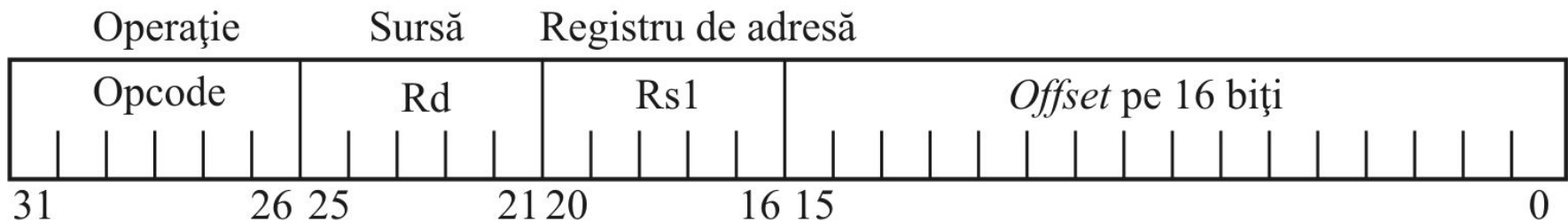
Daca indexul are valoarea 0 se obține load/store cu adresare indirectă
(caz particular)

Formatul instrucțiunii la procesoarele RISC

3. Formatul registru – memorie (R-M)



a) *Load*

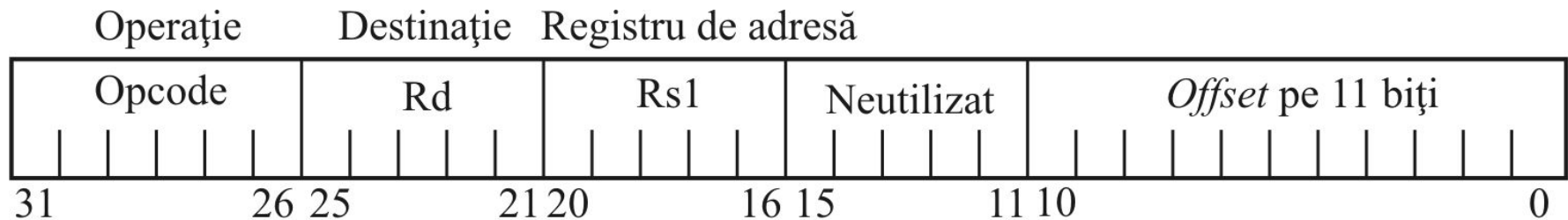


b) *Store*

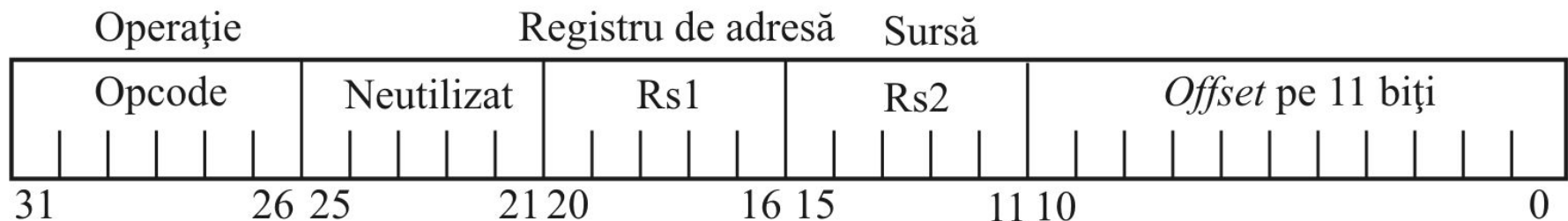
Formatul *load – store* (versiunea 1)

Formatul instrucțiunii la procesoarele RISC

3. Formatul registru – memorie (R-M)



a) *Load*



b) *Store*

Formatul *load – store* (versiunea 2)

Formatul instrucțiunii la procesoarele RISC

4. Formatul instrucțiunilor *branch* (salturi conditionate)

Construcție specifică limbajelor de nivel înalt (punct de decizie):

```
if((x!=y)&&(z<0))  
{  
  a=b-3 ;  
  b=b+4 ;  
}
```

Echivalența în cod masină:

```
if (x==y) goto L1;  
if (z>=0) goto L1;  
  a=b-3;  
  b=b+4;  
L1:
```

4. Formatul instructiunilor *branch* (salturi condiționate)

A. Utilizarea registrului de condiții (soluția CISC)

Registrul de condiții (flag-uri):

Z – zero = 1 - caracterizează rezultat zero

= 0 - caracterizează rezultat diferit de zero

C - *carry* = 1 - caracterizează rezultat cu transport (împrumut)

= 0 - caracterizează rezultat fără transport (împrumut)

S - *semn* = 1 - caracterizează rezultat negativ

= 0 - caracterizează rezultat pozitiv

O - *overflow* = 1 - caracterizează rezultat cu depășire

= 0 - caracterizează rezultat fără depășire

O listă tipică de instrucțiuni *branch*:

BL - *branch if less* – salt dacă este mai mic decât (<)

BG - *branch if greater* – salt dacă este mai mare decât (>)

BGE - *branch if greater or equal* – salt dacă este mai mare sau egal cu (>=)

BLE - *branch if less or equal* – salt dacă este mai mic sau egal cu (<=)

BE - *branch if equal* – salt dacă este egal cu (==)

BNE - *branch if not equal* – salt dacă nu este egal cu (!=)

Considerând că compilatorul alocă variabilelor x și y registrele R1 și respectiv R2, declarația:

if (x==y) goto L1

poate fi translatată într-o secvență de două instrucțiuni mașină:

CMP R1,R2 ; poziționează *flag*-urile în acord cu rezultatul scăderii R1-R2

BE L1 ; testează *flag*-ul z si execută saltul daca z=1.

4. Formatul instrucțiunilor *branch* (salturi condiționate)

B. Evitarea utilizării unui registru de condiții (soluția RISC)

Presupune utilizarea unor instrucțiuni de forma:

BEQ R1,R2,L1 ; salt la L1 dacă $R1 = R2$

BL R1,R2,L1 ; salt la L1 dacă $R1 < R2$

Cele două instrucțiuni (BEQ și BL) sunt suficiente pentru a implementa toate tipurile de test necesare limbajelor de nivel înalt:

Condiție	Secvența de cod aferentă
$R1 < R2$	BL R1,R2,L1
$R1 > R2$	BL R2,R1,L1
$R1 \geq R2$	BL R2,R1,L1 BEQ R1,R2,L1
$R1 \leq R2$	BL R1,R2,L1 BEQ R1,R2,L1
$R1 == R2$	BEQ R1,R2,L1
$R1 \neq R2$	BL R1,R2,L1 BL R2,R1,L1

Pentru a elimina secvențele de două instrucțiuni, sunt necesare încă două instrucțiuni:

BGE - salt dacă este mai mare sau egal cu ...

BNE - salt dacă nu este egal cu ...

4. Formatul instructiunilor *branch* (salturi condiționate)

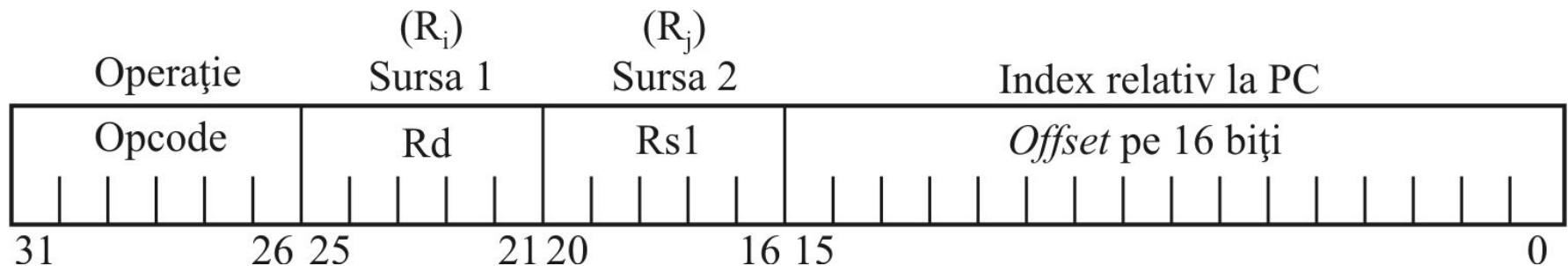
C. Opțiunea noastră privind instrucțiunile *branch*:

BEQ Ri,Rj,L1

BNE $R_i, R_j, L1$

BL Ri,Rj,L1

BGE $R_i, R_j, L1$



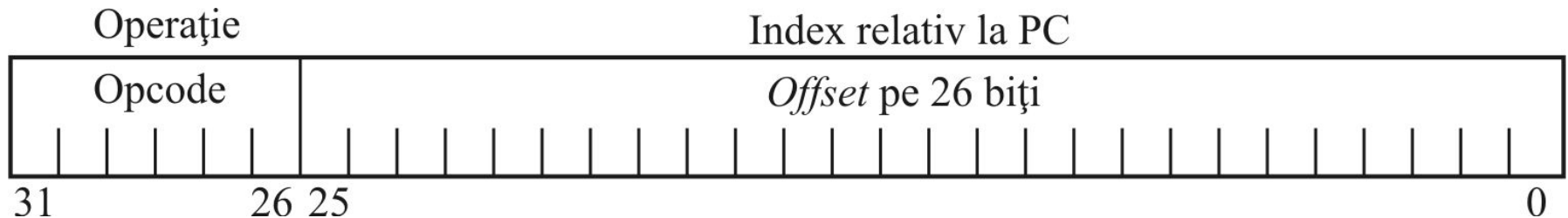
Formatul instrucțiunilor de *branch* (format de tip R-R-I)

5. Formatul instrucțiunilor *jump* (salturi necondiționate)

Salturile directe:

JMP L1 ; salt la adresa L1

Adresa de salt L1 se obține adunând la PC *offset*-ul codificat în codul instrucțiunii (adresare relativă la PC, ca în cazul instrucțiunilor *branch*).



Formatul instrucțiunii *jump*-direct (formatul I)

